

DTU



Material non-linearity (Day 2)

Casper Schousboe Andreasen & Ole Sigmund

Department of Civil and Mechanical Engineering, Solid Mechanics

Preparatory reading:

Handout Day 2

Chapter 17-17.3 in Cook et al.

Important points from Day 1

- Total number of DOFs (Degree of Freedom):
 - Bar 2D element: 2 dofs per node $\rightarrow 2 \cdot \text{nodes}$
 - Bar 3D element: 3 dofs per node $\rightarrow 3 \cdot \text{nodes}$
- Mapping from local to global DOFs
 - Bar 2D element have 4 dofs (2 nodes):
 - Associated global degree of freedom:

$$\text{edof} = \{2 \cdot \text{node1} - 1 \quad 2 \cdot \text{node1} \quad 2 \cdot \text{node2} - 1 \quad 2 \cdot \text{node2}\}$$
 - E.g. bar from node 5 to 6: $\{d\} = \{u_5 \quad u_5 \quad u_6 \quad u_6\}^T$

$$\text{edof} = \{2 \cdot 5 - 1 \quad 2 \cdot 5 \quad 2 \cdot 6 - 1 \quad 2 \cdot 6\} = \{9 \quad 10 \quad 11 \quad 12\}$$
 - In terms of the data structure in Matlab code:

$$\text{edof} = [\quad 2 \cdot \text{IX}(e, 1) - 1 \quad , \quad 2 \cdot \text{IX}(e, 1) \quad , \quad 2 \cdot \text{IX}(e, 2) - 1 \quad , \quad 2 \cdot \text{IX}(e, 2) \quad]$$

$$\text{edof} = [2*IX(e,1)-1 , 2*IX(e,1) , 2*IX(e,2)-1 , 2*IX(e,2)]$$

Three-bar truss example

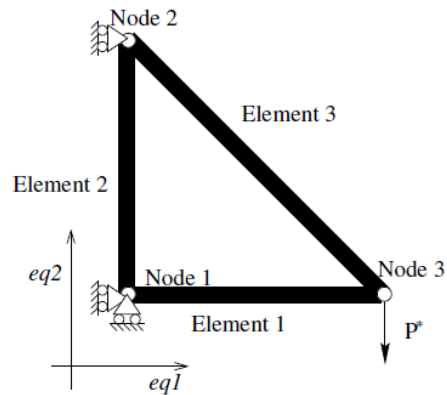


Figure 1.2: Numbering scheme for three-bar truss.

```
% Topology matrix IX(node1,node2,propno),
IX = [ 1  3  1
       1  2  1
       2  3  1 ];
```

edof is an array of length 4 (when working with 2D bar elements) and is used to **convert local DOF's to global DOF's** for each element e

to (1.13) can be illustrated by

$$\begin{bmatrix} \square + \triangle & \square + \triangle & \triangle & \triangle & \square & \square \\ \square + \triangle & \square + \triangle & \triangle & \triangle & \square & \square \\ \triangle & \triangle & \triangle + \diamond & \triangle + \diamond & \diamond & \diamond \\ \triangle & \triangle & \triangle + \diamond & \triangle + \diamond & \diamond & \diamond \\ \square & \square & \diamond & \diamond & \square + \diamond & \square + \diamond \\ \square & \square & \diamond & \diamond & \square + \diamond & \square + \diamond \end{bmatrix} \begin{Bmatrix} u_1 \\ v_1 \\ u_2 \\ v_2 \\ u_3 \\ v_3 \end{Bmatrix} - \begin{Bmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \\ p_5 \\ p_6 \end{Bmatrix} = \begin{Bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{Bmatrix} \quad (1.16)$$

> BLACKBOARD

ToDo before you make today's exercises

Clean up your code ... you will need to make a loop around most of it today

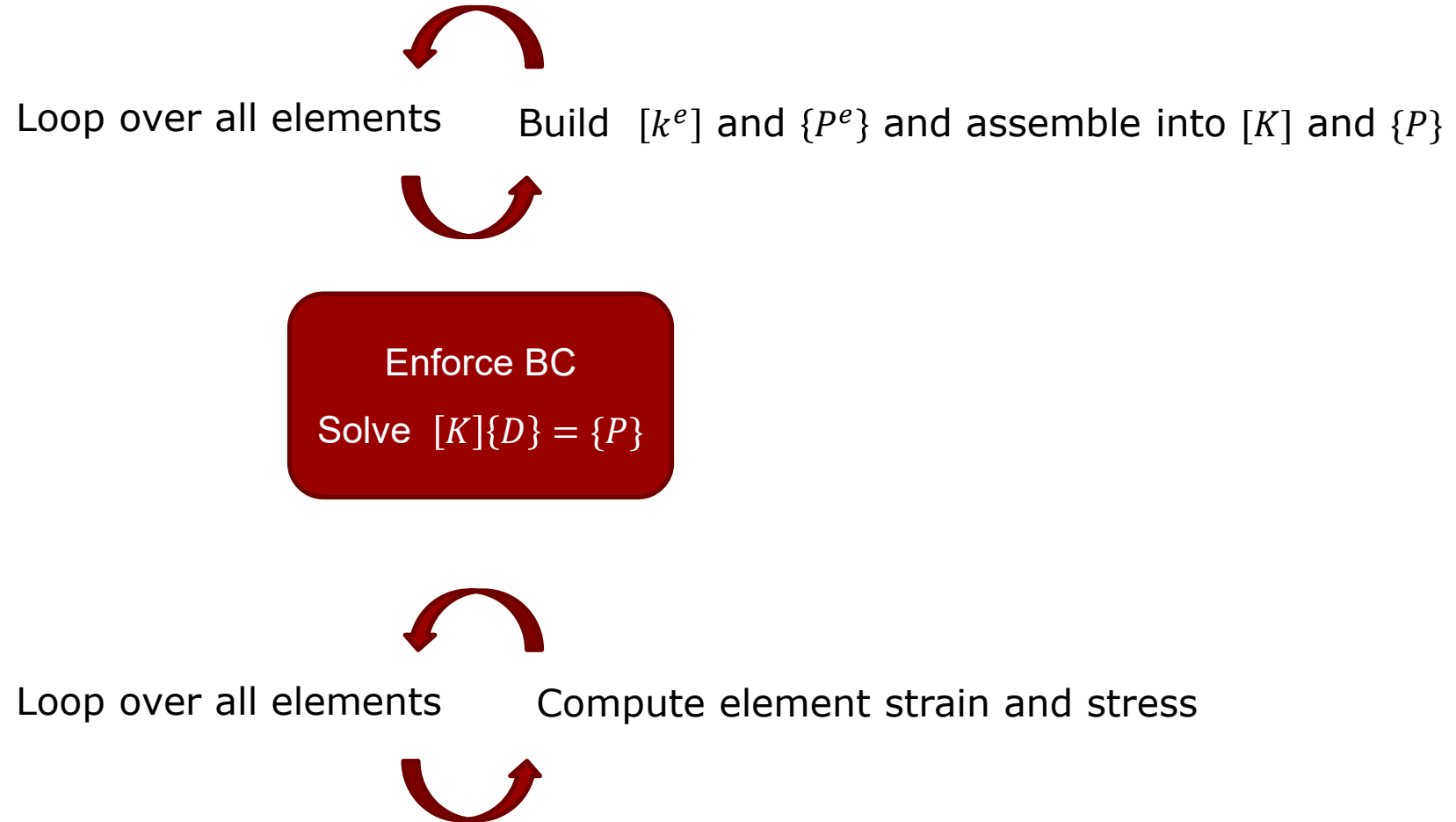
Input file for inputs... makes program general

Move constants and other specifics to input or preamble

Write a few comments and make sure names are descriptive

Recap: What is the structure of an FE code?

Linear
analysis:



Linear vs. non-linear analysis

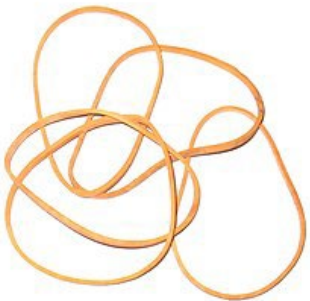
The main difference between the linear and the non-linear analysis is that the ***stiffness of the structure is not assumed to be constant through the simulation***



The sources of nonlinearity are classified into three main categories

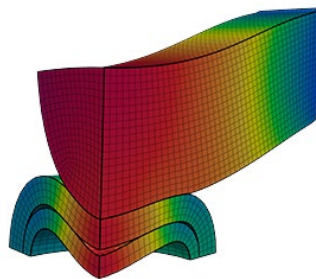
Material

Material non-linearity might include plasticity and creep



Boundary

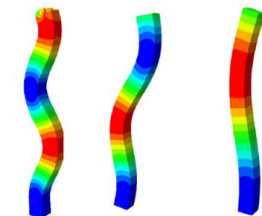
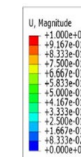
Boundary nonlinearity is most commonly associated with the modeling of contact surfaces and gaps or crack propagation.



Geometric

Geometric nonlinearities attributed to large displacements and/or to large strains.

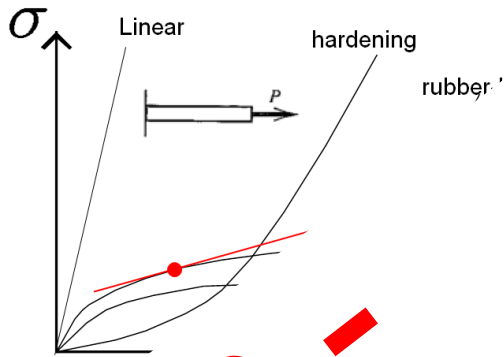
Geometric nonlinearity is e.g. important, when modeling the post-buckling behavior of a structure.



P_{cr1} = 118.5(N) P_{cr2} = 69.7(N) P_{cr3} = 28.2(N)

Examples of non-linearities

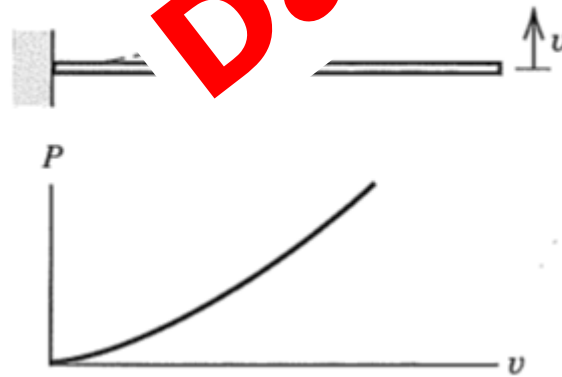
Material



Geometric

Geometric nonlinearities attributed to large displacements and/or to large strains.

Geometric nonlinearity is e important, when modeling buckling behavior of



Boundary

Boundary nonlinearity is most commonly associated with the modeling of contact surfaces or gaps.



Day 2 -
Today

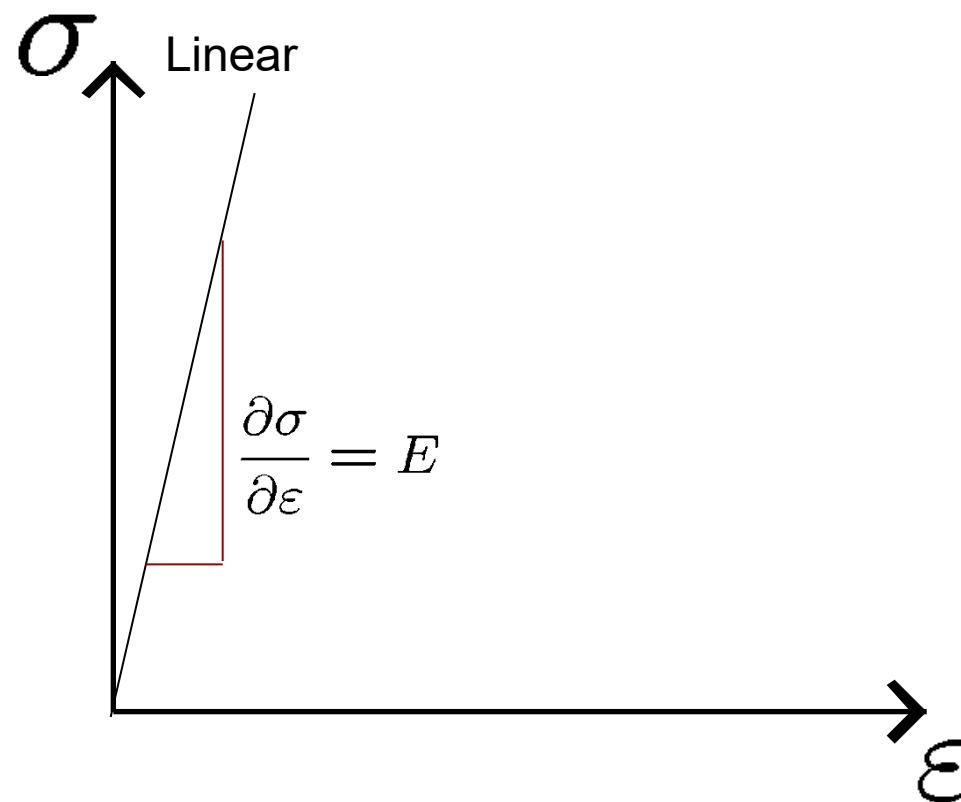
Day 3

Project?

Linear materials:

$$\sigma = E \varepsilon$$

$$\frac{\partial \sigma}{\partial \varepsilon} = E$$



Materials Behavior

Linear materials:

$$\sigma = E \varepsilon$$

$$\frac{\partial \sigma}{\partial \varepsilon} = E$$

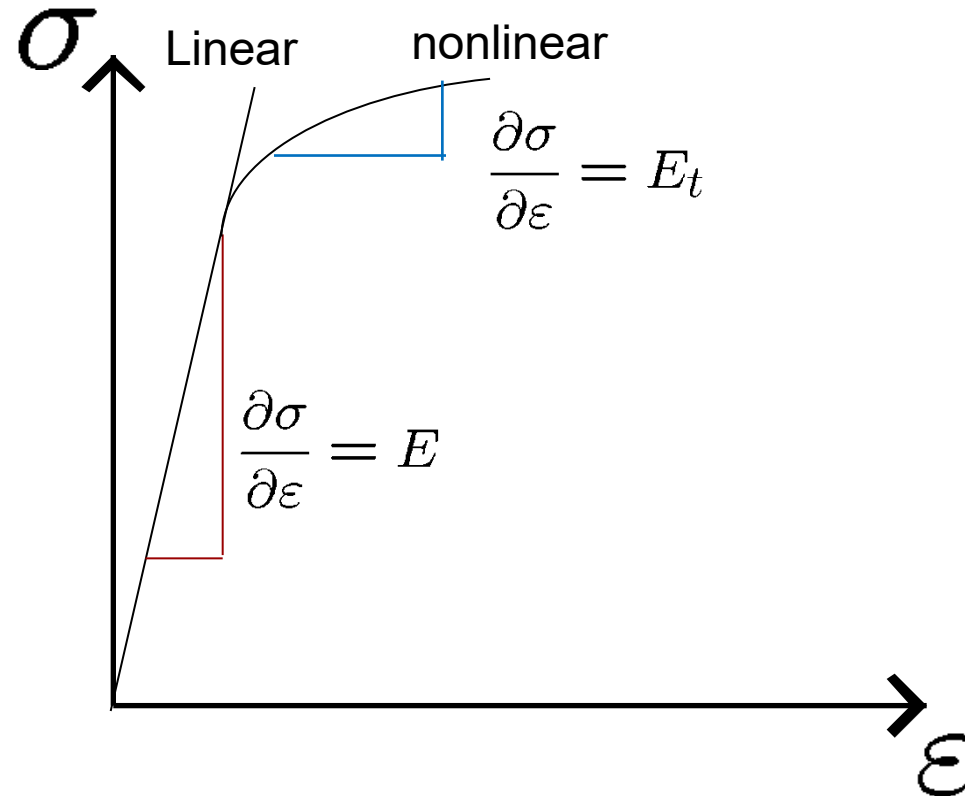
Nonlinear (elastic) materials:

$$\frac{\partial \sigma}{\partial \varepsilon} = E_t$$

Slope of σ - ε curve

⇓

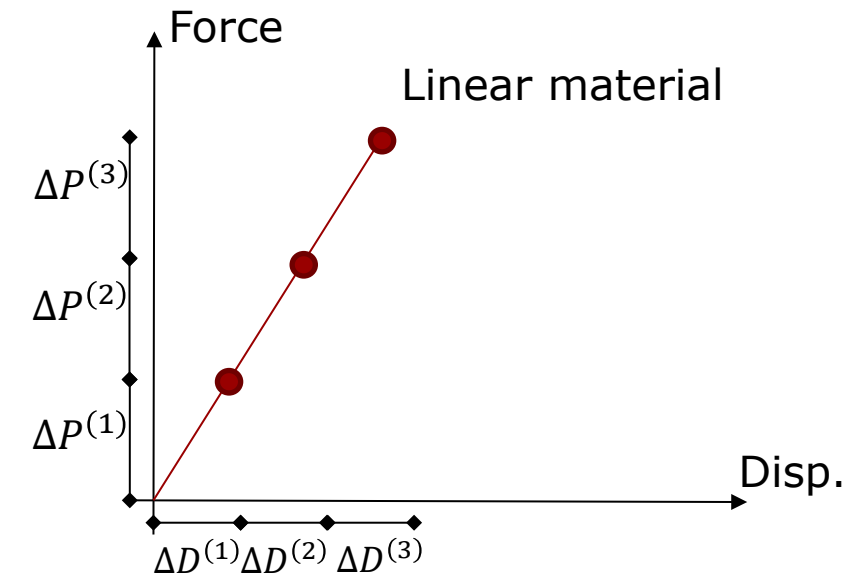
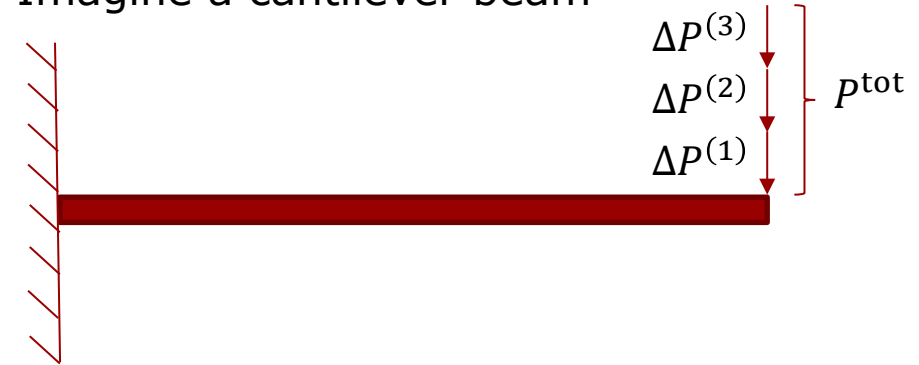
$$\partial \sigma = E_t \partial \varepsilon \quad \text{(Please note: } \sigma = E \varepsilon \text{ no longer apply to non-linear materials!)}$$



Incremental Method

- Apply full load, P^{tot} , increments: $P^{\text{tot}} = \sum_{n=1}^{\text{nincr}} P^n$
- Load at increment n : $P^n = P^{n-1} + \Delta P^n$
- Displacement at increment n $D^n = D^{n-1} + \Delta D^n$
- Increment size : $\Delta P^n, \Delta D^n$

Imagine a cantilever beam



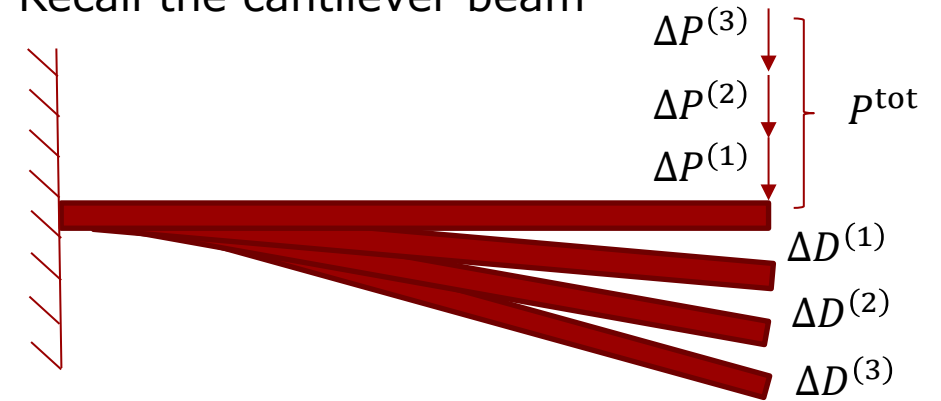
Incremental Procedures

- Explicit methods (No iterations – (forward)Euler methods)
Two examples:
 - Euler Method (Forward Euler)
 - Euler Method with one-step correction
- Implicit methods (equilibrium w. Newton-Raphson iterations)
Two examples:
 - Newton-Raphson Method (NR)
 - Modified NR Method (MNR)

Euler Method

Solution at increment n :

Recall the cantilever beam



$$[K_t]^{n-1} \{\Delta D\} = \{\Delta P\}$$

Imagine you apply an external load increment

Remember BC's !

⇓

Calculate displacement increments

$$\{D\}^n = \{D\}^{n-1} + \{\Delta D\}$$

Add displacement increments to previous displacements

Note: For non-linear material behaviour we do not update the geometry (small displacements). We will do that next week (large displacements)

Pseudo-code (Euler Method)

$$\begin{aligned} P^0 &= 0 \\ \Delta P^n &= P^{final} / n_{incr} \\ D^0 &= 0 \end{aligned}$$

Important to remember

How do we get the tangent stiffness?

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

Calculate $K_t(D^{n-1})$

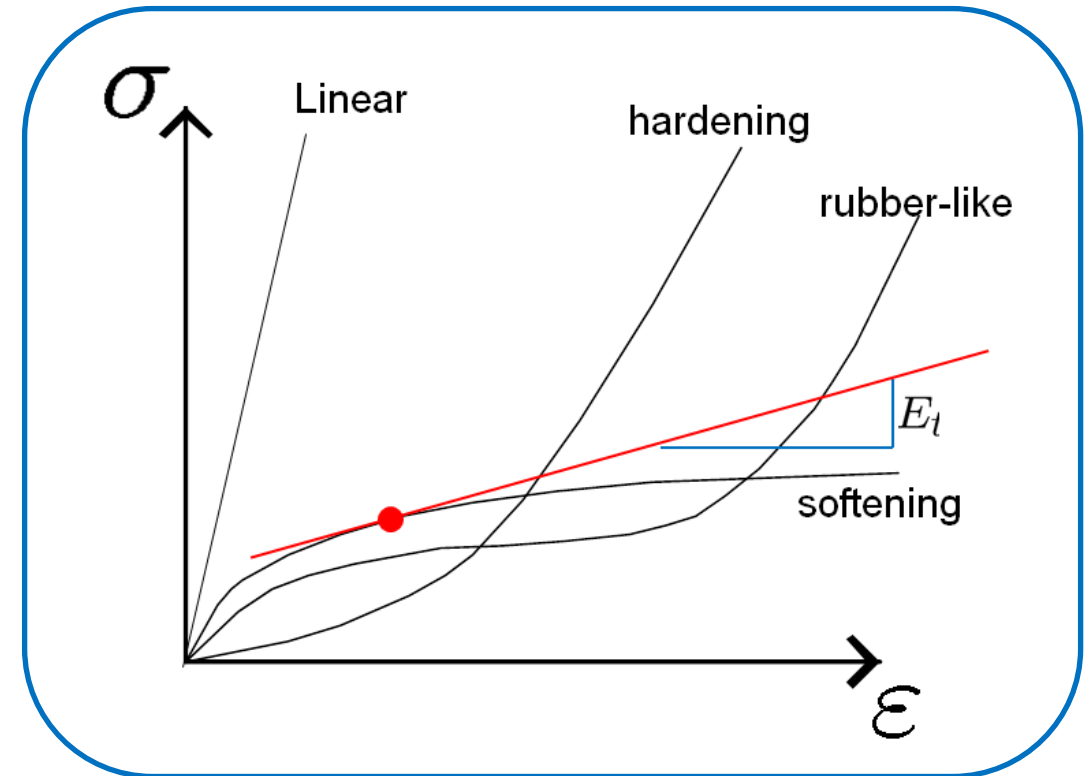
Enforce boundary conditions on K_t and ΔP^n

$$\Delta D^n = (K_t(D^{n-1}))^{-1} \Delta P^n$$

$$D^n = D^{n-1} + \Delta D^n$$

End load-increments

where n_{incr} is the number of load increments.



Tangent Stiffness Matrix

Definition:

$$[K_t] = \frac{\partial \{R\}}{\partial \{D\}} = \frac{\partial \{R_{int}\}}{\partial \{D\}} \quad (\text{Recall that (1.10); } \{R\} = \{R_{int}\} - \{R_{ext}\})$$

constant

$$= \sum_e \{B_0\} N^e L_0^e - \{P\} \quad \text{with: } N^e = A^e \sigma(\epsilon)$$

Bar element and nonlinear material:

$$[K_t] = \frac{\partial \{R_{int}\}}{\partial \{D\}} = \frac{\partial}{\partial \{D\}} \left(\sum_e \{B_0\} N^e L_0^e \right) \quad (\text{Recall that (1.7); } \{B_0\} = \frac{1}{L_0^2} \{-\Delta x, -\Delta y, \Delta x, \Delta y\}^T)$$

with $\{B_0\}$ evaluated at element level.

$$= \sum_e \{B_0\} \frac{\partial N^e}{\partial \{d\}} L_0^e = \sum_e \{B_0\} \frac{\partial \sigma}{\partial \{d\}} A^e L_0^e$$

$$= \sum_e \{B_0\} \frac{\partial \sigma}{\partial \epsilon} \frac{\partial \epsilon}{\partial \{d\}} A^e L_0^e = \sum_e \{B_0\} \frac{\partial \sigma}{\partial \epsilon} \{B_0\}^T A^e L_0^e \quad (\text{by use of definition (1.4)})$$

Tangent Stiffness Matrix

Local *tangent* stiffness matrix:

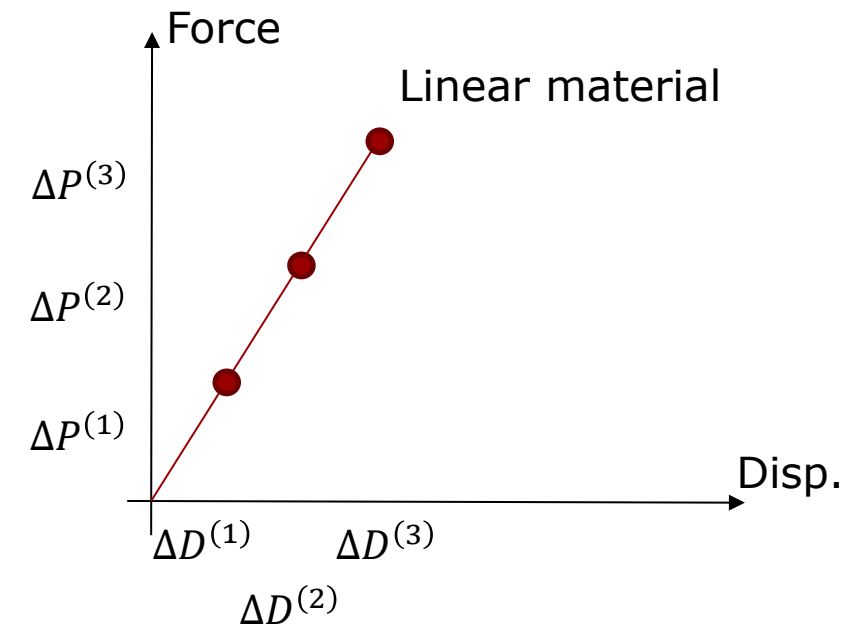
$$[k_t] = \frac{\partial \sigma}{\partial \varepsilon} A^e L_0^e \{B_0\} \{B_0\}^T$$

Local stiffness matrix (for comparison, see (1.15)):

$$[k] = E^e A^e L_0^e \{B_0\} \{B_0\}^T$$

Linear material:

$$\frac{\partial \sigma}{\partial \varepsilon} = E \quad \Rightarrow \quad \text{Tangent stiffness matrix} = \text{stiffness matrix}$$



MAKE A LINEAR TEST CASE

$$P^0 = 0$$

$$\Delta P^n = P^{final} / n_{incr}$$

$$D^0 = 0$$

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

Calculate $K_t(D^{n-1})$

Enforce boundary conditions on K_t and ΔP^n

$$\Delta D^n = (K_t(D^{n-1}))^{-1} \Delta P^n$$

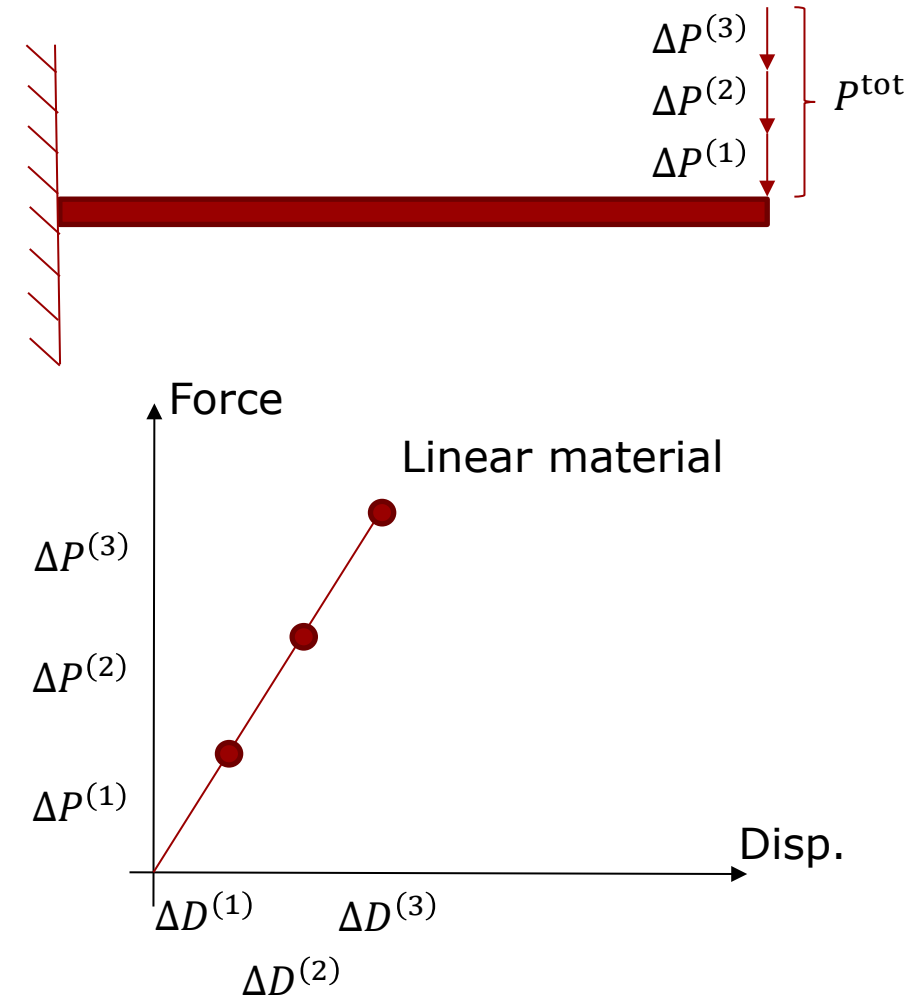
$$D^n = D^{n-1} + \Delta D^n$$

End load-increments

where n_{incr} is the number of load increments.

If the material (problem) is linear solving ANY FE-problem in n_{incr} increments each with $\Delta P^{(n)}$ will result in exactly the same solution as if it was solved directly by P^{tot} .

If not: the solution algorithm is not correctly implemented.



Euler Method with One-Step Correction

Solution at increment n : Imagine you apply an external load increment

$$[K_t]^{n-1} \{\Delta D\} = \{\Delta P\} - \{R\}^{n-1}$$

Calculate displacement increments

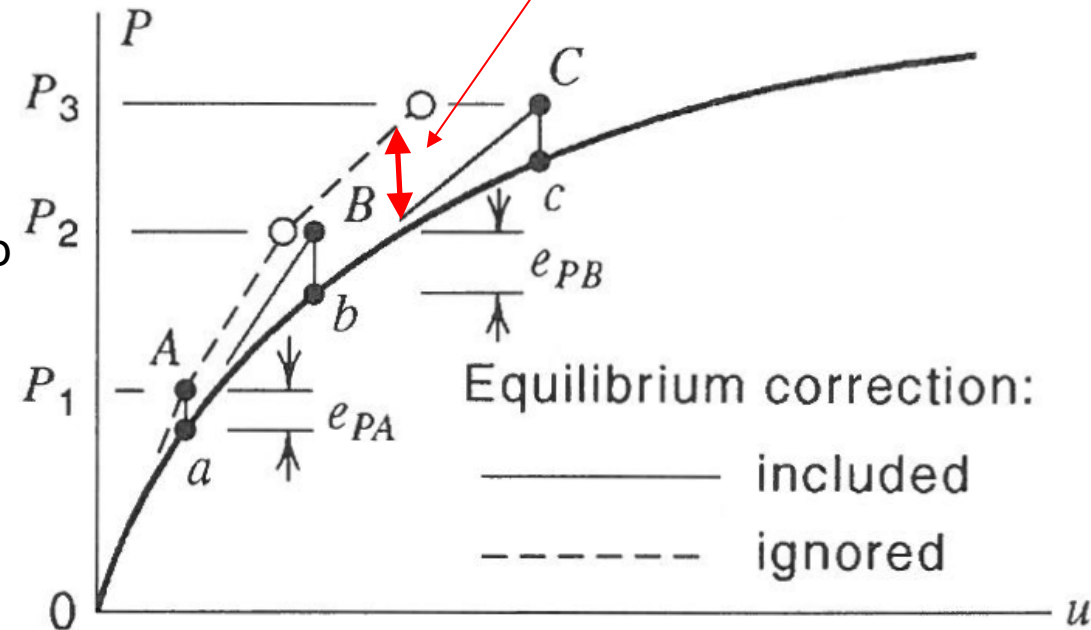


Subtract residual from previous step

$$\{D\}^n = \{D\}^{n-1} + \{\Delta D\}$$

Add displacement increments

The solution is drifting



Cook, Fig. 17.2-5

Pseudo-code (Euler Method with One-Step Correction)

$$P^0 = 0$$

$$\Delta P^n = P^{final} / n_{incr}$$

$$D^0 = 0$$

$$R^0 = 0$$

Important to remember

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

Calculate $K_t(D^{n-1})$

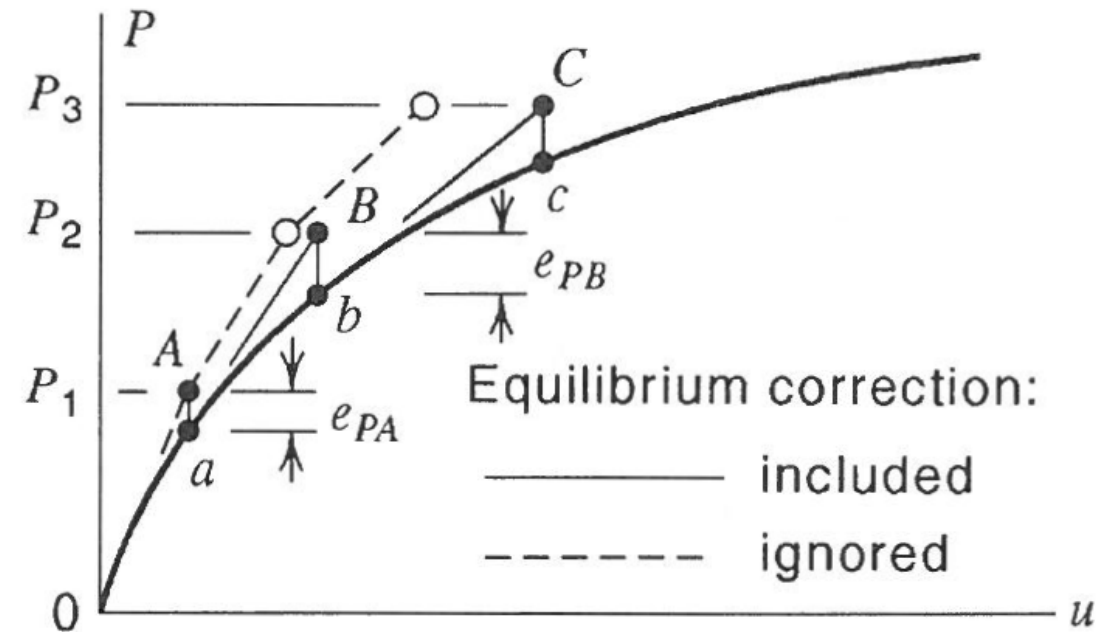
Enforce boundary conditions on K_t and $(\Delta P^n - R^{n-1})$

$$\Delta D^n = (K_t(D^{n-1}))^{-1}(\Delta P^n - R^{n-1})$$

$$D^n = D^{n-1} + \Delta D^n$$

$$\text{Calculate } R^n = R_{int}^n(D^n) - P^n$$

End load-increments



Cook, Fig.
17.2-5

What is $R_{int}(D)$?

- For linear materials: $\{R\} = \{R_{int}\} - \{R_{ext}\} = \{0\} = [K]\{D\} - \{P\}$
- So $\{R_{int}\}$ are the internal forces
- For non-linear material trusses we see from (2.11) and (2.12):

As in Chapter 1, the residual for truss elements is written as

$$\{R\} = \{R_{int}\} - \{R_{ext}\} = \left[\sum_e \{B_0\} N^e L_0^e \right] - \{P\} = \{0\}, \quad (2.11)$$

where the element forces are

$$N^e = A^e \sigma(\epsilon) \quad (2.12)$$

Newton Raphson Method

Solution at increment n :

$$[K_t]^{n-1} \{\Delta D\} = \{\Delta P\}$$

$\Downarrow$

$$\{D\}_1^n = \{D\}^{n-1} + \{\Delta D\}$$

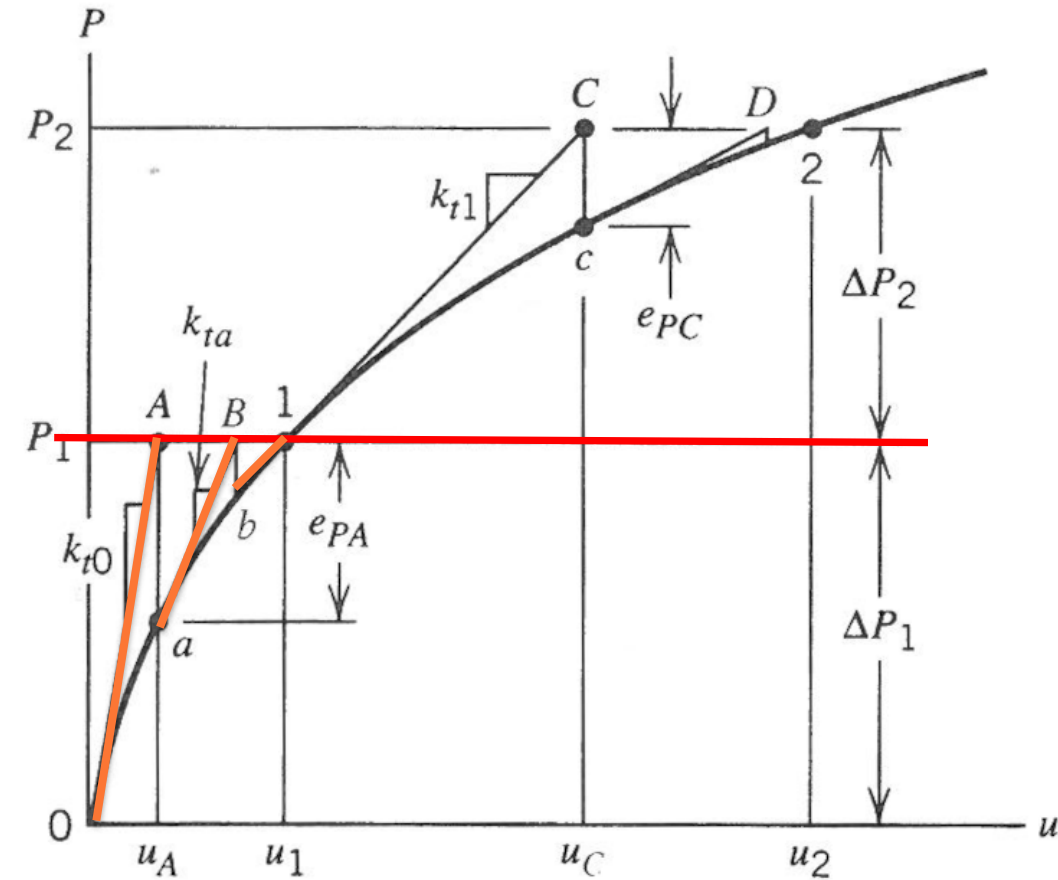
Remember BC

Iteration until convergence ($R=0$):

$$[K_t]_i^{n-1} \{\Delta D\}_i = -\{R\}_i^n$$

$\Downarrow$

$$\{D\}_i^n = \{D\}_{i-1}^n + \{\Delta D\}_i$$



Cook, Fig. 17.2-2a

Pseudo-code (Newton-Raphson Method)

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

$$D_0^n = D^{n-1}$$

For equilibrium iterations $i = 0, 1, \dots, i_{max}$

$$\text{Calculate } R_i^n = R_{int}(D_i^n) - P^n$$

$$\text{Stop iterations when } ||R_i^n|| \leq \varepsilon_{stop} ||P^n||$$

$$\text{Calculate } K_t(D_i^n) \text{ (and factorize)}$$

$$\text{Solve equilibrium equation } \Delta D_i^n = -(K_t(D_i^n))^{-1} R_i^n \quad (\text{NB! Remember BC})$$

$$\text{Update displacements } D_{i+1}^n = D_i^n + \Delta D_i^n \cdot \alpha$$

End equilibrium iterations

$$D^n = D_i^n$$

End load-increments

Page 10 in the notes:

In order to stabilize convergence of the NR-algorithm, one may introduce a damping factor in the displacement update, i.e. $D_{i+1}^n = D_i^n + \alpha \Delta D_i^n$, where $\alpha \in [0, 1]$. The damping coefficient α may be chosen constant, from a line-search or adaptively.

Newton Raphson Method

However, it is relatively expensive!

- need to factorize $K_t(D_i^n)$ for every iteration

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

$$D_0^n = D^{n-1}$$

For equilibrium iterations $i = 0, 1, \dots, i_{max}$

$$\text{Calculate } R_i^n = R_{int}(D_i^n) - P^n$$

$$\text{Stop iterations when } ||R_i^n|| \leq \epsilon_{stop} ||P^n||$$

$$\text{Calculate } K_t(D_i^n) \text{ (and factorize)}$$

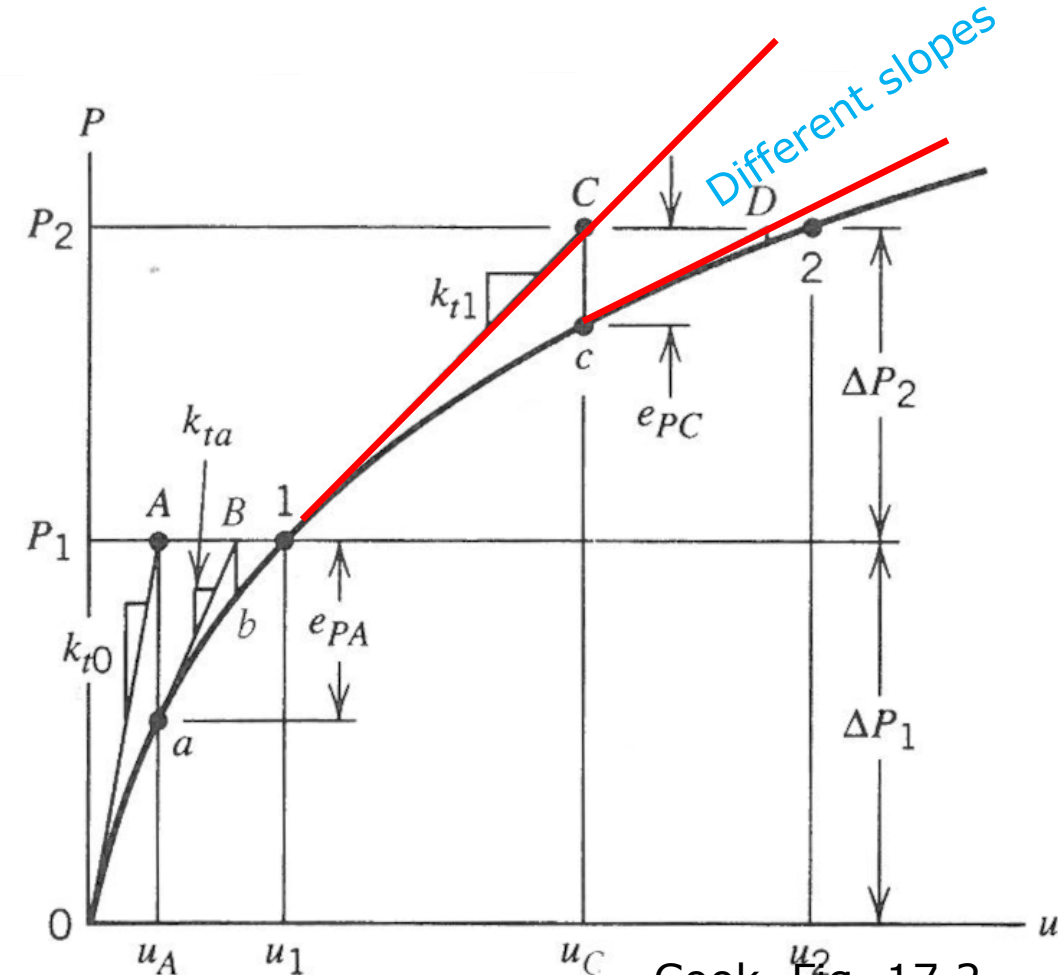
$$\text{Solve equilibrium equation } \Delta D_i^n = -(K_t(D_i^n))^{-1} R_i^n$$

$$\text{Update displacements } D_{i+1}^n = D_i^n + \Delta D_i^n$$

End equilibrium iterations

$$D^n = D_i^n$$

End load-increments



Cook, Fig. 17.2-2a

$$\begin{aligned} [K_t]^{n-1} \{\Delta D\} &= \{\Delta P\} \\ \Downarrow \\ \{D\}_1^n &= \{D\}^{n-1} + \{\Delta D\} \end{aligned}$$
$$\boxed{[K_t]^{n-1}} \{\Delta D\}_i = -\{R\}_i^n$$

$$\Downarrow$$

$$\{D\}_i^n = \{D\}_{i-1}^n + \{\Delta D\}_i$$
$$\{D\}_i^n = \{D\}_{i-1}^n + \{\Delta D\}_i$$


Cook, Fig. 17.2-2b

Factorization why do we need to care?

Solving a linear equation system contains two operations:

- Factorization (LU) → Obtain a triangular matrix(two), $\frac{2}{3}n^3$ flops
- Back substitution, n^2 flops

What is a flop?

Floating Point Operations

For symmetric positive definite matrices faster methods exist one example:
Cholesky Factorization, $\frac{1}{3}n^3$ flops and half memory (due to symmetry).

Commands for solving:

Matlab: `D = K_corr \ P`

NumPy: `D = np.linalg.solve(K_corr, P)`

SciPy: `D = sp.sparse.linalg.spsolve(K_corr, P)`

Commands for factorizing:

Matlab: `C = chol(K_corr);`

`[L,U] = lu(K_corr);`

NumPy: `lu = np.linalg.splu(K_corr)`

SciPy: `lu = sp.sparse.linalg.splu(K_corr)`

back substitution:

`D = C \ (C \ P)`

`D = U \ (L \ P)`

`D = lu.solve(P)`

`D = lu.solve(P)`

Pseudo-code (Modified NR-Method)

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

$$D_0^n = D^{n-1}$$

Calculate $K_t(D_0^n)$ (and factorize)

Use LU-factorization in Matlab (see p. 10 in the lecture notes).

Cholesky factorization might be even faster and consume less memory! (Only applies for Symmetric Positive Definite systems)

For equilibrium iterations $i = 0, 1, \dots, i_{max}$

$$\text{Calculate } R_i^n = R_{int}(D_i^n) - P^n$$

Stop iterations when $\|R_i^n\| \leq \epsilon_{stop} \|P^n\|$

Solve equilibrium equation $\Delta D_i^n = -(K_t(D_0^n))^{-1} R_i^n$ (NB! Remember BC)

$$\text{Update displacements } D_{i+1}^n = D_i^n + \Delta D_i^n$$

End equilibrium iterations

$$D^n = D_i^n$$

End load-increments

MAKE A LINEAR TEST CASE

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

$$D_0^n = D^{n-1}$$

For equilibrium iterations $i = 0, 1, \dots, i_{max}$

$$\text{Calculate } R_i^n = R_{int}(D_i^n) - P^n$$

Stop iterations when $\|R_i^n\| \leq \epsilon_{stop} \|P^n\|$

Calculate $K_t(D_i^n)$ (and factorize)

$$\text{Solve equilibrium equation } \Delta D_i^n = -(K_t(D_i^n))^{-1} R_i^n$$

$$\text{Update displacements } D_{i+1}^n = D_i^n + \Delta D_i^n$$

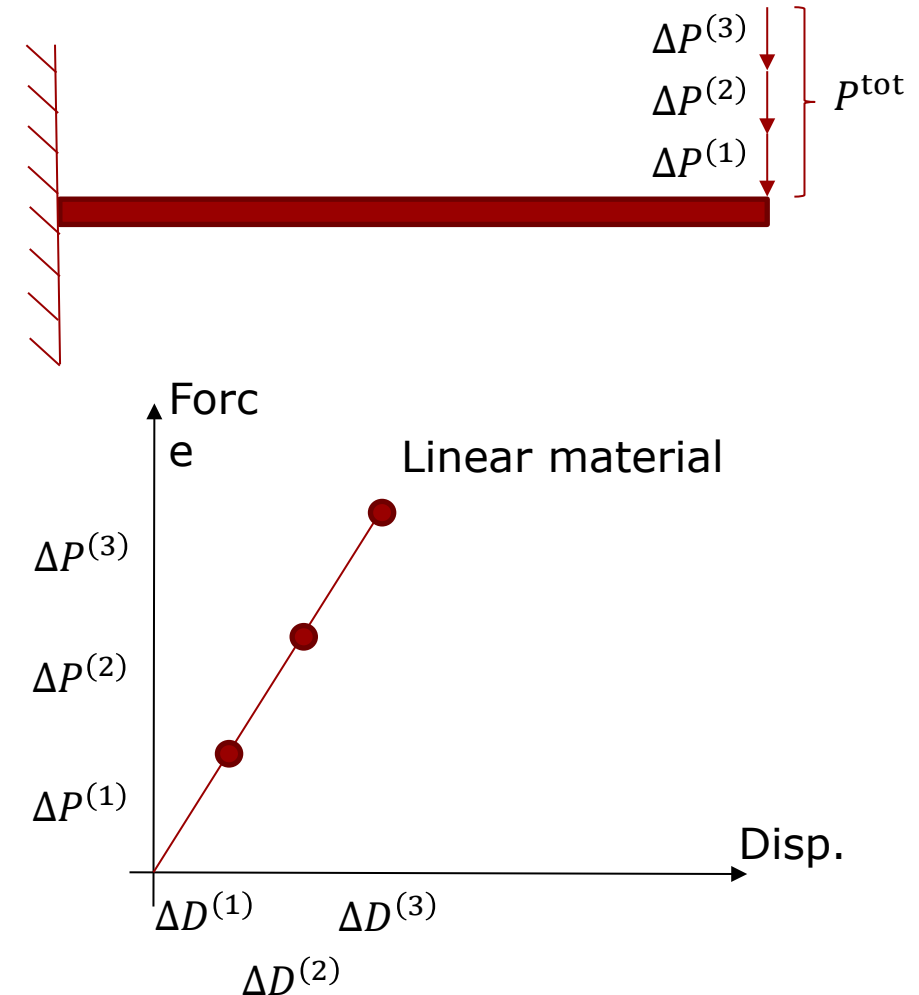
End equilibrium iterations

$$D^n = D_i^n$$

End load-increments

If the material (problem) is linear solving ANY FE-problem in n_{incr} increments each with $\Delta P^{(n)}$ and iterations will result in exactly the same solution as if it was solved directly by P^{tot} .

If not: the solution algorithm is not correctly implemented.



Recap

Euler Method

$$P^0 = 0$$

$$\Delta P^n = P^{final} / n_{incr}$$

$$D^0 = 0$$

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

Calculate $K_t(D^{n-1})$

Enforce boundary conditions on K_t and ΔP^n

$$\Delta D^n = (K_t(D^{n-1}))^{-1} \Delta P^n$$

$$D^n = D^{n-1} + \Delta D^n$$

End load-increments

Euler Method with one-step equilibrium-correction

$$P^0 = 0$$

$$\Delta P^n = P^{final} / n_{incr}$$

$$D^0 = 0$$

$$R^0 = 0$$

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

Calculate $K_t(D^{n-1})$

Enforce boundary conditions on K_t and $(\Delta P^n - R^{n-1})$

$$\Delta D^n = (K_t(D^{n-1}))^{-1} (\Delta P^n - R^{n-1})$$

$$D^n = D^{n-1} + \Delta D^n$$

$$\text{Calculate } R^n = R_{int}^n(D^n) - P^n$$

End load-increments

Recap

Newton-Raphson algorithm

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

$$D_0^n = D^{n-1}$$

For equilibrium iterations $i = 0, 1, \dots, i_{max}$

$$\text{Calculate } R_i^n = R_{int}(D_i^n) - P^n$$

Enforce boundary conditions on R_i^n

Stop iterations when $\|R_i^n\| \leq \epsilon_{stop} \|P^{final}\|$

$$\text{Calculate } K_t(D_i^n)$$

Enforce boundary conditions on $K_t(D_i^n)$

$$\text{Solve equilibrium equation } \Delta D_i^n = -(K_t(D_i^n))^{-1} R_i^n$$

$$\text{Update displacements } D_{i+1}^n = D_i^n + \Delta D_i^n$$

End equilibrium iterations

$$D^n = D_i^n$$

End load-increments

Euler Method with one-step equilibrium-correction

$$P^0 = 0$$

$$\Delta P^n = P^{final} / n_{incr}$$

$$D^0 = 0$$

$$R^0 = 0$$

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

$$\text{Calculate } K_t(D^{n-1})$$

Enforce boundary conditions on K_t and $(\Delta P^n - R^{n-1})$

$$\Delta D^n = (K_t(D^{n-1}))^{-1} (\Delta P^n - R^{n-1})$$

$$D^n = D^{n-1} + \Delta D^n$$

$$\text{Calculate } R^n = R_{int}^n(D^n) - P^n$$

End load-increments

Recap

Newton-Raphson algorithm

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

$$D_0^n = D^{n-1}$$

For equilibrium iterations $i = 0, 1, \dots, i_{max}$

$$\text{Calculate } R_i^n = R_{int}(D_i^n) - P^n$$

Enforce boundary conditions on R_i^n

Stop iterations when $\|R_i^n\| \leq \epsilon_{stop} \|P^{final}\|$

$$\text{Calculate } K_t(D_i^n)$$

Enforce boundary conditions on $K_t(D_i^n)$

$$\text{Solve equilibrium equation } \Delta D_i^n = -(K_t(D_i^n))^{-1} R_i^n$$

$$\text{Update displacements } D_{i+1}^n = D_i^n + \Delta D_i^n$$

End equilibrium iterations

$$D^n = D_i^n$$

End load-increments

Modified Newton-Raphson algorithm

For load-increment $n = 1, 2, \dots, n_{incr}$

$$P^n = P^{n-1} + \Delta P^n$$

$$D_0^n = D^{n-1}$$

Calculate $K_t(D_0^n)$

Enforce boundary conditions on $K_t(D_0^n)$

Factorize $K_t(D_0^n)$

For equilibrium iterations $i = 0, 1, \dots, i_{max}$

$$\text{Calculate } R_i^n = R_{int}(D_i^n) - P^n$$

Enforce boundary conditions on R_i^n

Stop iterations when $\|R_i^n\| \leq \epsilon_{stop} \|P^{final}\|$

$$\text{Solve equilibrium equation } \Delta D_i^n = -(K_t(D_0^n))^{-1} R_i^n$$

$$\text{Update displacements } D_{i+1}^n = D_i^n + \Delta D_i^n$$

End equilibrium iterations

$$D^n = D_i^n$$

End load-increments

Non-linear elastic Analysis

Typical nonlinear (large deformation) material for rubber

The Signorini stress-strain relation

$$\sigma(\epsilon) = c_1 (\lambda - \lambda^{-2}) + c_2 (1 - \lambda^{-3}) + c_3 (1 - 3\lambda + \lambda^3 - 2\lambda^{-3} + 3\lambda^{-2})$$

The stretch

$$\lambda = 1 + c_4 \epsilon$$

Tangent stiffness

$$E_t(\epsilon) = \frac{d\sigma}{d\epsilon} = c_4 [c_1 (1 + 2\lambda^{-3}) + 3c_2 \lambda^{-4} + 3c_3 (-1 + \lambda^2 - 2\lambda^{-3} + 2\lambda^{-4})]$$

Elastic-Plastic Analysis

Typical nonlinear material is metal plasticity

Do you think the presented methods represent metal behavior accurately?

Elastic unloading requires a model;

```

 $\sigma_{max} = \sigma_Y$ 
if  $|\sigma| < \sigma_{max}$  then
     $E_{ep} = E$  (Elastic)
else
     $E_{ep} = E_t$  (Plastic)
     $\sigma_{max} = |\sigma|$ 
end
(isotropic hardening)
  
```

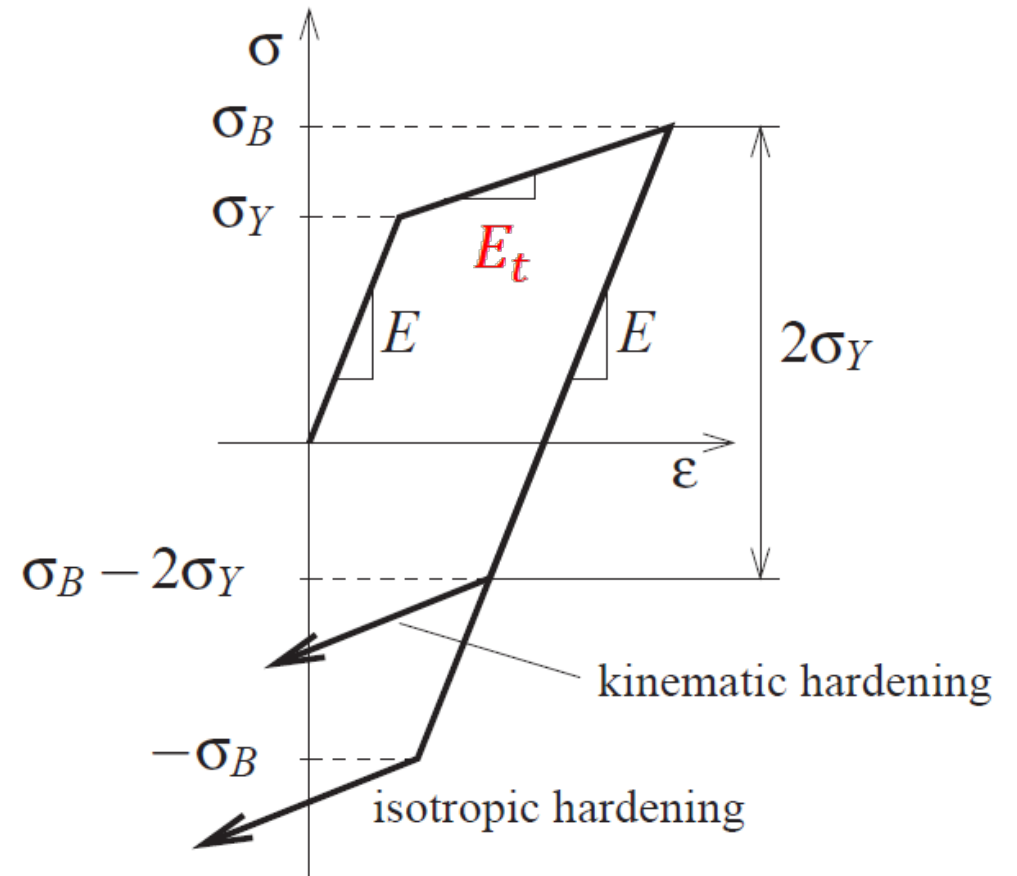
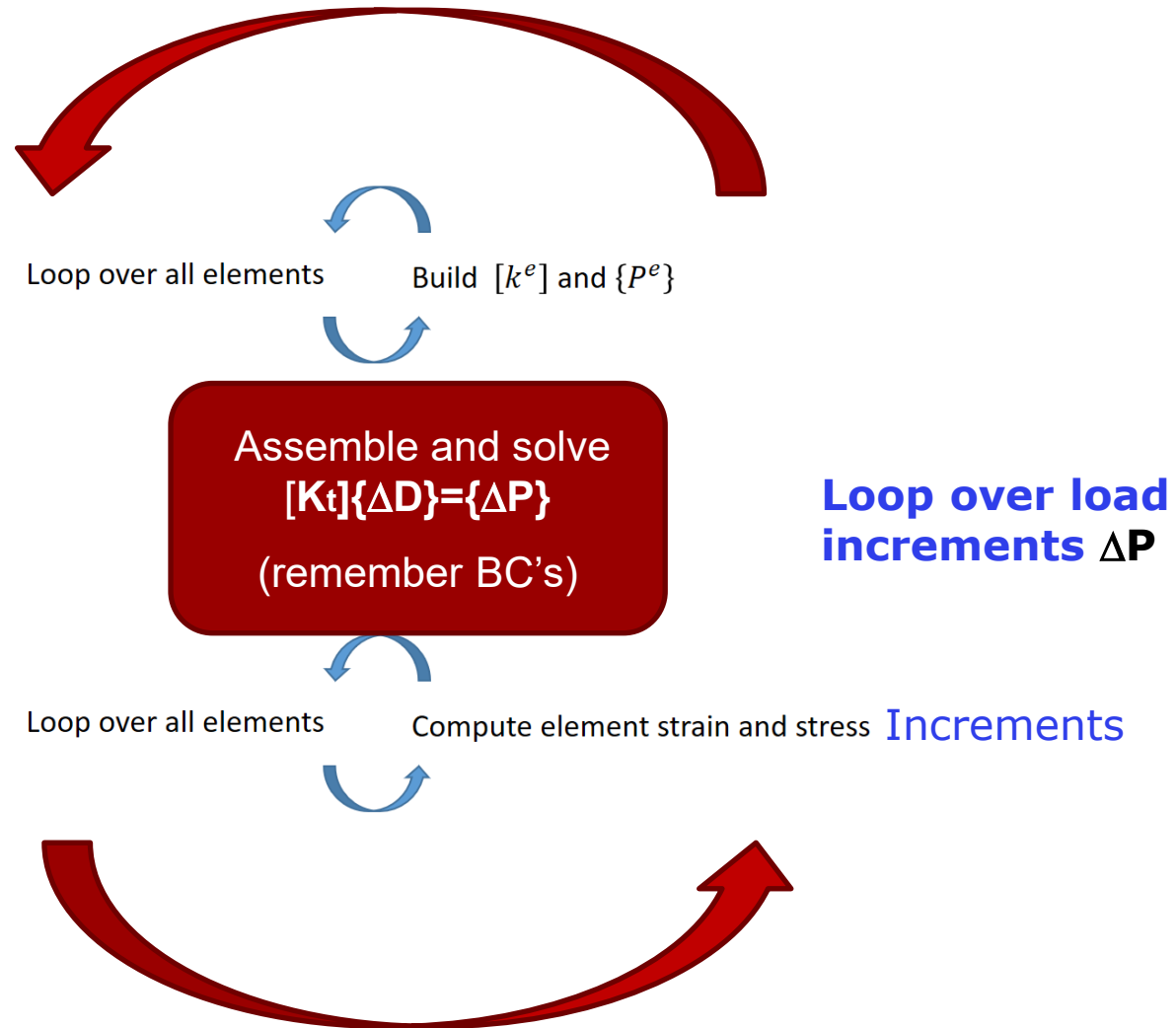


Figure 2.1: Isotropic and kinematic hardening.

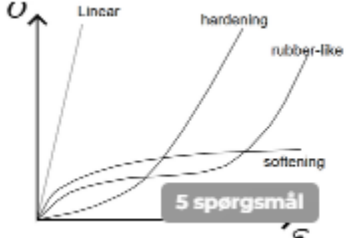
What is the structure of a FE code?

Non-linear



PlayTime

kahoot...



41525 FEM - Day 02

5 spørgsmål

KAHOOT__olsi190

0 runder

Tildel

Præsenter live

Exercises

Hints:

$$\text{edof} = [2 \cdot \text{IX}(e,1)-1, 2 \cdot \text{IX}(e,1), 2 \cdot \text{IX}(e,2)-1, 2 \cdot \text{IX}(e,2)]$$

- Before you start today's exercises, try to simplify and clean up your code from last time (for example, use the "edof=[...]" command as discussed in the lecture) This will make today's extensions easier.
- Make a new directory called "DAY2" (under the "FEM" directory) and copy example files and truss program to this directory.
- Keep backup copies of old files that work!

Rubber is a typical example of a non-linear material. The Signorini stress-strain relation that mimics rubber response can in 1D be written as

$$\sigma(\epsilon) = c_1 (\lambda - \lambda^{-2}) + c_2 (1 - \lambda^{-3}) + c_3 (1 - 3\lambda + \lambda^3 - 2\lambda^{-3} + 3\lambda^{-2}), \quad (2.21)$$

where $\lambda = 1 + c_4 \epsilon$ is the stretch.

The tangent stiffness modulus for this material can be found by differentiation

$$E_t(\epsilon) = \frac{d\sigma}{d\epsilon} = c_4 [c_1 (1 + 2\lambda^{-3}) + 3c_2 \lambda^{-4} + 3c_3 (-1 + \lambda^2 - 2\lambda^{-3} + 2\lambda^{-4})]. \quad (2.22)$$

Note that this semi-artificial material model only works for $\lambda > 0$, i.e. $\epsilon > -1/c_4$. If the strain gets below this value in one of your examples, you should decrease the load.

Exercise 2.1

For later check of your FE-code, start by writing a small Matlab script that plots the force as a function of displacement for a bar of length 3 and area 2 made of the rubber-like material. Use the material constants $c_1 = 1$, $c_2 = 50$, $c_3 = 0.1$, $c_4 = 100$ and a maximum force of 200 to get started.

Implement the Euler method for truss structures with the rubber like material law (2.21) and (2.22). Plot the force-displacement curve of a straight truss structure (Figure 2.2) for different increment sizes and compare them with the analytical curve. To get started, you may use the values, $A = 2$, $P^{final} = 200$, 20 load increments and therefore $\Delta P = 10$. The number of load increments (n_{incr}) and the final load (P^{final}) should be given in the input file.



Figure 2.2: Simple 2-bar test specimen.

Exercise 2.2

Implement the "incremental with one-step equilibrium-correction" method for the 2-bar truss structure with the same non-linear material law as above. *Plot the force displacement curve for the simple 2-bar truss for different increment sizes and compare them with the analytical curve and the curves for the pure Euler method.*

Exercise 2.3

Implement an incremental scheme with NR-method equilibrium iterations and *test it on the 2-bar problem like in the previous exercises*. The maximum (NR) equilibrium iterations (e.g. $i_{\max}=100$) should be given in the input file.

Exercise 2.4

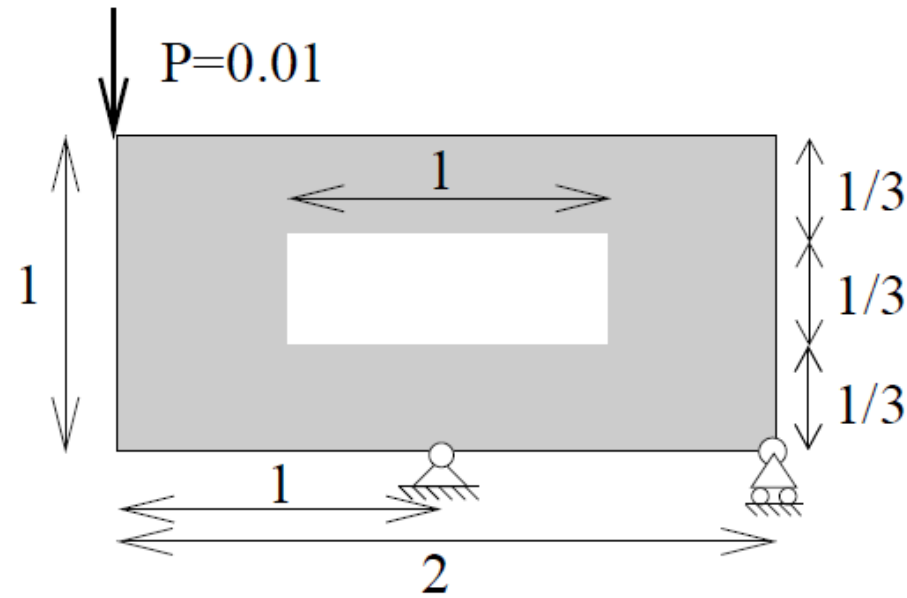
Implement the modified NR-method and *test it on the 2-bar problem like in the previous exercises*. Remember that you can factorize the stiffness matrix once pr. load step and then save time (for larger problems) by doing simple forward-backward-substitution for each iteration.

*Exercise 2.5

Implement the (pure) incremental method for elastic-plastic analysis (with loading and unloading) and *compare results for isotropic and kinematic hardening models*. *For illustrative purposes make the elastic elements blue and the plastic ones red.*

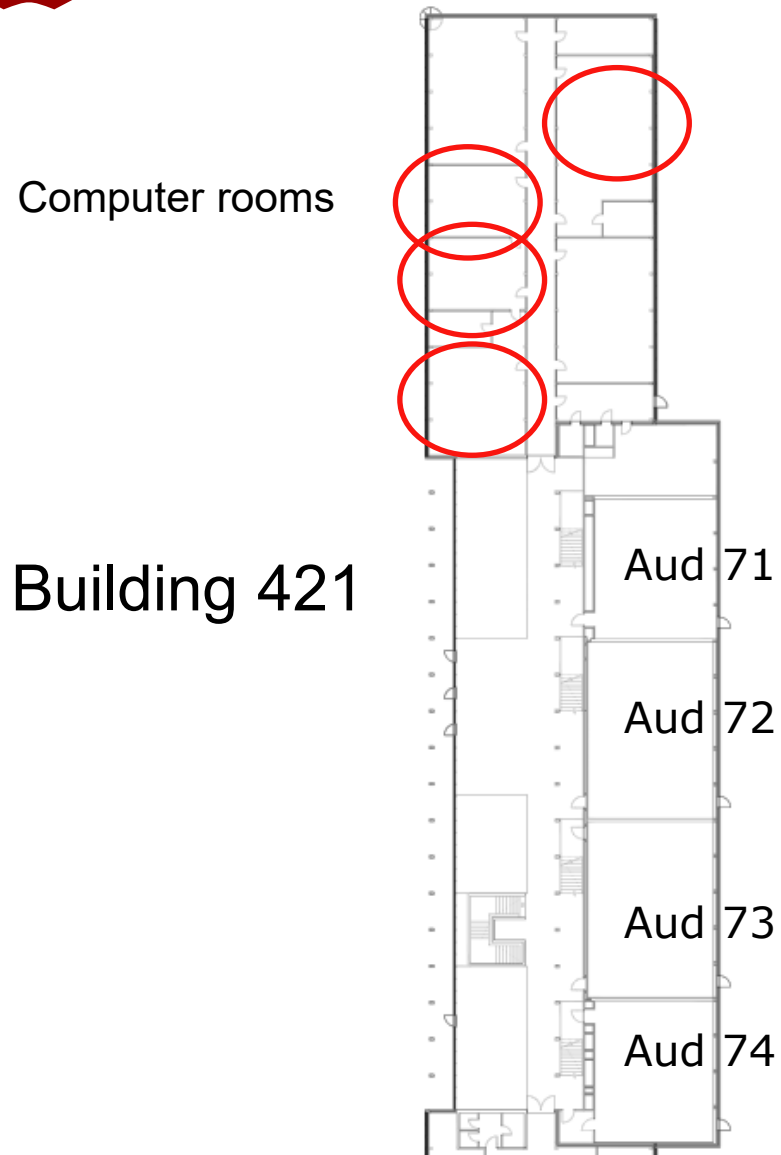
Competition time!

Here is a competition: Who can make the stiffest truss structure? This corresponds to the truss structure with the smallest displacement (in the direction of the load) where the load is applied! ¹



¹The submission (not the ranking!) is part of the evaluation of the first report in the course!

Computer exercises



- Distribute in computer rooms
- Organize groups ! Students without a group please get in contact with me.
- Read notes/exercises
- Use notes, book, brain, other students – then teachers / TAs ...

Good luck and have fun!!